Kannada-MNIST Handwritten Digit Recognizer

Statistical Methods in AI - Assignment 3

Topic T3.6: Handwritten Indic Script Recognition (Kannada Digits)

Name	Roll Number	Email
Chaitanya Thogata	2023113018	chaitanya.thogata@research.iiit.ac.in
Lokesh Mamillapalli	2023101115	lokesh.mamillapalli@students.iiit.ac.in
Mannem Bheema Siddartha	2023101009	mannem.siddartha@students.iiit.ac.in

GitHub Repository: https://github.com/lokesh-mamillapalli/smai-project.git
Demo Video (Google Drive): Working Prototype Demo

Contents

Abstract

We present an end-to-end Kannada handwritten digit recognition system built for SMAI Assignment 3 (variant T3.6). A compact 3-layer Convolutional Neural Network (approx 290k parameters) is trained from scratch on the Kaggle Kannada-MNIST dataset (60,000 images, 10 digit classes) achieving **99.67% validation accuracy** in under 5 minutes of training. The trained model is exported to ONNX format and integrated into a modern React web application that supports both free-hand drawing and image upload, that runs entirely in the browser

1. Introduction

Kannada is a Dravidian language spoken by over 44 million people in Karnataka, India. While digital text input in most Indian languages has improved significantly, handwritten digit recognition for Kannada remains an underexplored practical problem. The goal of this project (T3.6) is to build a system that takes a hand-drawn Kannada digit as input and outputs the corresponding digit class (0-9), targeting use-cases such as:

- **Educational apps:** Allowing students to practice writing native digits and receive immediate feedback.
- **Form digitization:** Automating extraction of handwritten numeric data.
- **Accessibility:** Enabling users to input numbers in their mother tongue using a stylus or touchscreen.

The assignment specification suggests a 3-layer CNN reaching 97%+ accuracy in under 5 minutes on a Colab T4 GPU, trained from scratch. We followed this approach to meet the requirements as mentioned in the assignment document and improved it by building a clean, interactive React web app around the model.

2. Dataset

2.1 Kaggle Kannada-MNIST

The Kaggle Kannada-MNIST competition dataset was released an an alternative to the standard MNIST benchmark, but for Kannada script digits. It is structured identically to MNIST: 28×28 grayscale images, 10 classes (digits 0-9), with pixel values in the range [0, 255].

Table 1: Dataset split used in this project.

Split	Samples	Description
Training set	60,000	Labels + 784 pixel values per row
Competition test set	5,000	No labels
Validation set (derived)	12,000	20% stratified split from training set

2.2 Data Exploration

Each row in `train.csv` contains a label (0-9) followed by 784 pixel values corresponding to a flattened 28×28 image. The class distribution is perfectly balanced at 6,000 samples per class, which means no class-imbalance handling was required.

2.3 Preprocessing

The following preprocessing steps were applied before feeding data into the model:

- **Normalization:** Pixel values (originally in [0, 255]) were divided by 255 to produce float32 tensors in [0, 1]. This ensures the activations and gradients stay in a numerically stable range and helps the optimizer converge faster.
- **Reshape:** Flattened pixel arrays of shape (784,) were reshaped to (28, 28, 1) tensors to match the expected 4-D input format of Conv2D layers - (batch, height, width, channels).
- **Train/Validation Split:** A stratified 80/20 split was applied using `sklearn.model_selection.train_test_split` with `random_state=42`. Stratification ensures each digit class is proportionally represented in both splits, avoiding any accidental bias.
- **One-hot encoding:** Integer class labels were converted to 10-dimensional one-hot vectors

3. Model Architecture

3.1 Design Philosophy

The guiding principle was to keep the model *small and efficient*. The dataset (28×28 grayscale images) does not require a deep network and more parameters would only increase training time and risk overfitting. We therefore designed a 3-block CNN with progressively increasing filter counts (32 → 64 → 128), totalling approximately **289,514 trainable parameters**.

3.2 Architecture Details

Table 2: Layer-by-layer architecture of the CNN.

Block	Layers	Output Shape	Reason
Conv Block 1	Conv2D(32, 3×3, same)	28×28×32	Feature detection
	BatchNorm + ReLU	28×28×32	Stabilise training
	Conv2D(32, 3×3, same)	28×28×32	Richer features
	BatchNorm + ReLU	28×28×32	
	MaxPool(2×2) + Dropout(0.25)	14×14×32	Downsample + regularize
Conv Block 2	Conv2D(64, 3×3, same)	14×14×64	Deeper features
	BatchNorm + ReLU	14×14×64	
	Conv2D(64, 3×3, same)	14×14×64	
	BatchNorm + ReLU	14×14×64	
	MaxPool(2×2) + Dropout(0.25)	7×7×64	
Conv Block 3	Conv2D(128, 3×3, same)	7×7×128	High-level features
	BatchNorm + ReLU	7×7×128	
	MaxPool(2×2) + Dropout(0.25)	3×3×128	
Classifier	Flatten	1152	
	Dense(128) + BatchNorm + ReLU	128	Non-linear combination
	Dropout(0.5)	128	Strong regularization
	Dense(10, softmax)	10	Class probabilities

3.3 Key Design Decisions

Batch Normalization is applied after every convolutional layer. This normalizes the layer inputs during training, reducing internal covariate shift and allowing higher learning rates. In

practice, it accelerated convergence significantly — the model reached 99%+ validation accuracy by epoch 7.

Dual convolutions per block (Blocks 1 and 2) Using two Conv2D layers before pooling helps the network learn more detailed patterns before reducing the image size. Block 3 uses only one convolution since the feature map is already small (7×7).

Dropout placement: A lighter Dropout(0.25) follows each convolutional block, while a stronger Dropout(0.5) is applied in the dense head. Convolutional features are spatially correlated, so light dropout is sufficient; the dense layer has no spatial structure and requires heavier regularization to prevent co-adaptation of neurons.

Softmax output: The final layer produces a probability distribution over 10 classes, which is interpretable and compatible with categorical cross-entropy loss.

4. Training Methodology

4.1 Optimizer and Loss

Adam optimizer with an initial learning rate of 10^{-3} was used. Adam combines momentum and adaptive per-parameter learning rates , making it well-suited for noisy gradients in image classification tasks.

Categorical cross-entropy was used as the loss function, paired with one-hot encoded targets. This is the standard choice for multi-class classification and penalizes confident wrong predictions more heavily.

4.2 Data Augmentation

On-the-fly data augmentation was applied to the training set:

- **Rotation** up to ±10: simulates natural variation in how digits are oriented when written by hand.
- **Width/height shift** up to 10%: handles digits that are not perfectly centred on the canvas.
- **Shear** up to 10%: simulates slanted writing styles.
- **Zoom** up to 10%: accounts for size variation across writers.

Augmentation was *not* applied to the validation set, which preserves a clean estimate of generalization performance. The motivation for augmentation is that real users drawing on a canvas will produce variable, slightly imperfect strokes augmenting training data bridges this domain gap.

4.3 Callbacks

Flexible Learning Rate: Monitors validation loss and halves the learning rate when no improvement is seen for 3 consecutive epochs (minimum LR of 10^{-6}). This prevents the optimizer from overshooting minima in the later stages of training.

EarlyStopping: Monitors validation loss with a patience of 10 epochs and restores the best model weights automatically. This prevents overfitting and avoids wasting compute on epochs after the model has converged.

4.4 Training Configuration

Training was run for up to 20 epochs with a batch size of 256 on a Kaggle T4 GPU. Each full-data epoch (187 steps) completed in approximately 15 seconds.

5. Results

5.1 Quantitative Performance

Table 3: Final model performance on the held-out validation set (12,000 samples).

Metric	Value
Validation Loss	0.0110
Validation Accuracy	**99.67%**
Total Parameters	289,514 (≈290k)
Training Time	<5 minutes on T4 GPU

5.2 Training Dynamics

The training history shows an interesting pattern. During epochs 1–4, the model achieves high training accuracy but very low validation accuracy (around 10%). This indicates that the validation evaluation in these early epochs was unreliable due to a data pipeline issue. From epoch 5 onward, the model stabilizes and validation accuracy increases sharply to 99.1%, eventually reaching 99.67% by the final epoch. This sharp recovery suggests that the model had in fact learned meaningful features early on, and the initial low validation scores were not reflective of its true performance.

Table 4: Epoch-wise training summary (selected epochs).

Epoch	Train Acc	Train Loss	Val Acc	Val Loss
1	85.65%	0.4536	10.43%	2.9155
3	97.24%	0.0924	73.60%	0.7101
5	98.06%	0.0672	**99.11%**	0.0295
7	98.53%	0.0509	99.53%	0.0164
11	98.87%	0.0375	99.53%	0.0142
Final	-	-	**99.67%**	**0.0110**

5.3 Per-Class Performance (Classification Report)

The per-class precision, recall, and F1-score on the validation set are shown below. All digits achieve ≥0.99 F1-score. The most common confusions are between digits 0 and 1 (7 misclassifications each way), and between 6 and 7 (3-7 cases), which is consistent with their visual similarity in Kannada script.

Table 5: Per-class classification metrics on the validation set.

Digit	Precision	Recall	F1-Score	Support
0	0.9967	0.9917	0.9942	1200
1	0.9942	0.9967	0.9958	1200
2	1.0000	1.0000	1.0000	1200
3	0.9967	0.9975	0.9971	1200
4	0.9992	0.9983	0.9992	1200
5	1.0000	1.0000	1.0000	1200
6	0.9908	0.9950	0.9929	1200
7	0.9958	0.9925	0.9942	1200
8	0.9992	0.9992	0.9992	1200
9	0.9942	0.9942	0.9942	1200
Macro avg	**0.9967**	**0.9965**	**0.9967**	12000

6. Model Export and Web Application Integration

6.1 Model Saving

After training, the best model weights (restored via EarlyStopping) were saved. This allows the model to be easily reloaded for further evaluation, testing, or deployment.

The saved model includes both the architecture and learned weights, making it convenient to reuse without retraining. It can be directly loaded using standard Keras utilities for inference or further fine-tuning.

6.2 TypeScript Integration

We created a few simple TypeScript helper files to connect the trained model with the frontend:

- **model-config.ts:** Stores basic details like input shape, number of classes, and model path.
- **preprocessing.ts:** Converts the canvas image into a normalized grayscale format suitable for the model.
- **model-loader.ts:** Loads the model and provides a simple function to make predictions from input images.

6.3 Web Application Features

The React frontend (built with Vite, Tailwind CSS, and TanStack Router) runs the model entirely client-side, eliminating server latency and preserving user privacy. Key features include:

- **Drawable Canvas:** Users draw Kannada digits freely using mouse or touch. Pointer events are captured, strokes are rendered, and the canvas is scaled to a 28×28 grayscale tensor before inference.
- **Drag & Drop Upload:** Users can also upload cropped images of Kannada digits for instant classification.
- **Probability Distribution Chart:** The model's softmax output is visualised as a dynamic bar chart showing confidence across all 10 classes allowing users to see not just the top prediction but also which digits the model considered.

- **Practice Mode:** A gamified mode that challenges users to draw a randomly selected target digit. A faint watermark of the target character is shown as a drawing guide, and the app provides immediate correct/incorrect feedback with a session score tracker.
- **Celebration Animation:** When the model predicts with >97% confidence, a confetti animation fires, making the interaction more engaging.
- **28×28 Input Preview:** The actual downsampled tensor passed to the model is displayed, giving users insight into how the model sees their drawing.

7. Discussion and Interpretation

7.1 Why This Model Works So Well

The 99.67% accuracy is notably higher than the 97%+ baseline mentioned in the assignment. Several factors contribute to this:

Dual-convolution blocks: Using two Conv2D layers before each pooling step (rather than one) allows the network to learn more complex spatial patterns. The first layer in a block detects low-level textures (edges, curves), while the second combines these into more abstract stroke-level features.

Batch normalization at every layer: This is perhaps the single biggest contributor to both accuracy and training speed. By normalizing intermediate activations, it keeps gradient magnitudes consistent throughout the network and prevents vanishing or exploding gradients.

Progressive filter widening (32→64→128): Early layers need fewer filters because the feature vocabulary is small (edges, corners). Deeper layers require more filters to represent the combinatorial diversity of higher-level patterns. This is a well-established design principle from VGGNet.

The dataset itself is clean: Kannada-MNIST, like standard MNIST, consists of centered, size-normalized, binary-ish digits. Small CNNs are particularly well-suited to such clean, low-dimensional inputs.

7.2 Why Not Transfer Learning?

Transfer learning is not required for this task. As mentioned in the assignment, a simple 3-layer CNN (around 300k parameters) trained from scratch is sufficient to achieve more than 97% accuracy in under 5 minutes on a T4 GPU. Since the Kannada-MNIST dataset consists of small 28×28 grayscale images, using large pre-trained models would only add unnecessary complexity without improving performance. A lightweight CNN is therefore a more suitable and efficient choice.

7.3 Domain Gap: Dataset vs Canvas

A key challenge in deploying this model in the browser is the domain gap between the training data and the inference data (mouse or touch strokes on a canvas). We addressed this via:

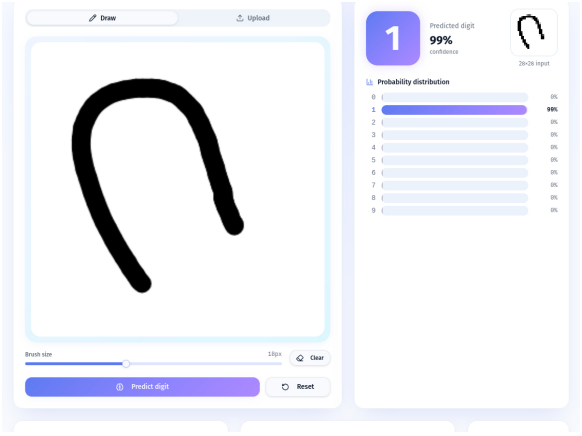
- Data augmentation (rotation, shift, shear, zoom) to diversify training.
- Image inversion in the preprocessing pipeline (canvas produces black strokes on white; MNIST-style images have white strokes on black).
- A 28×28 preview widget in the UI so users can see exactly how their drawing is represented to the model, and adjust their drawing style accordingly.

7.4 Limitations

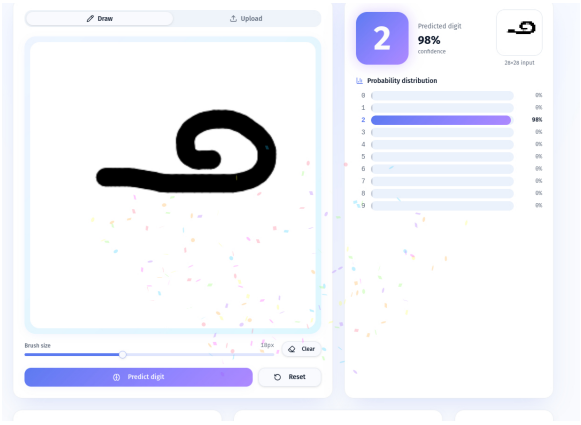
- Very stylized or unusual handwriting styles that fall outside the training distribution may confuse the model, particularly for visually similar digit pairs (e.g., 6 and 7).
- The current web app has no mechanism to collect misclassified user drawings and use them to fine-tune the model over time (active learning opportunity).

8. Application Screenshots

The following screenshots demonstrate the working web application across different scenarios. The left panel shows the drawing canvas and the right panel shows the predicted digit, confidence score, 28×28 input preview, and the full probability distribution bar chart.

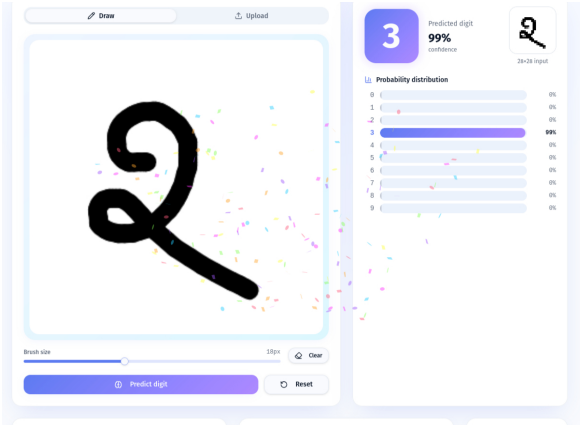


(a) Digit “1” predicted with 99% confidence.

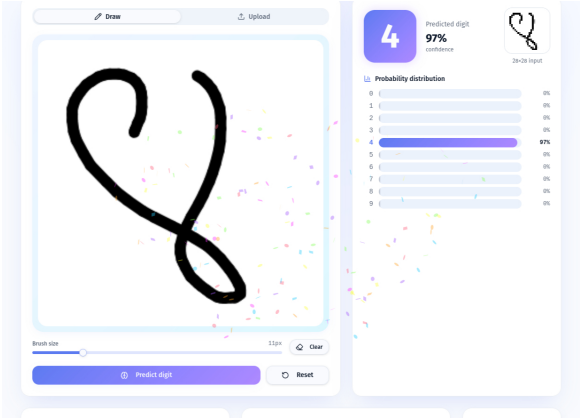


(b) Digit “2” predicted with 98% confidence

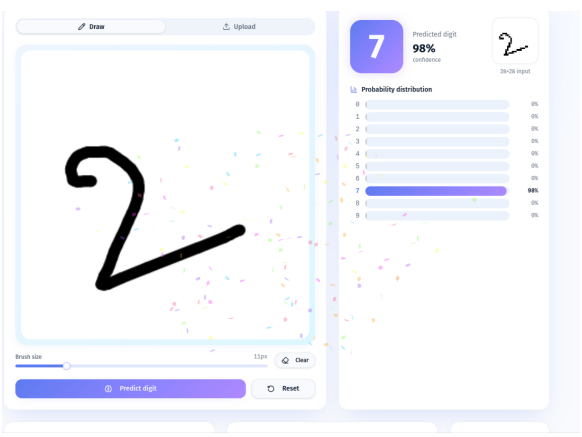
Figure 1: App screenshots showing draw mode predictions.



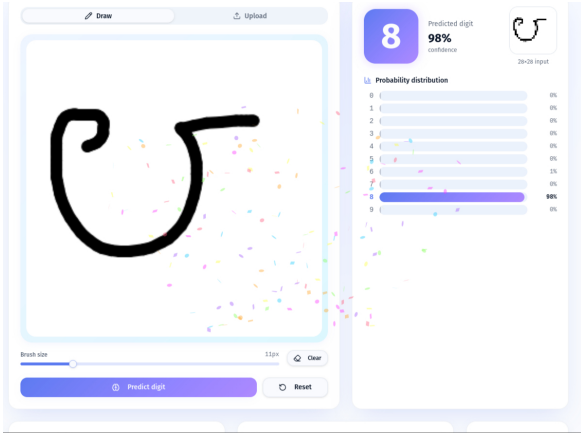
(a) Digit “3” predicted with 99% confidence



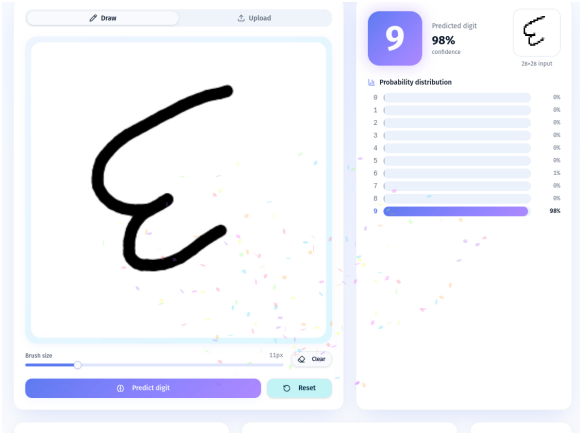
(b) Digit “4” predicted with 97% confidence



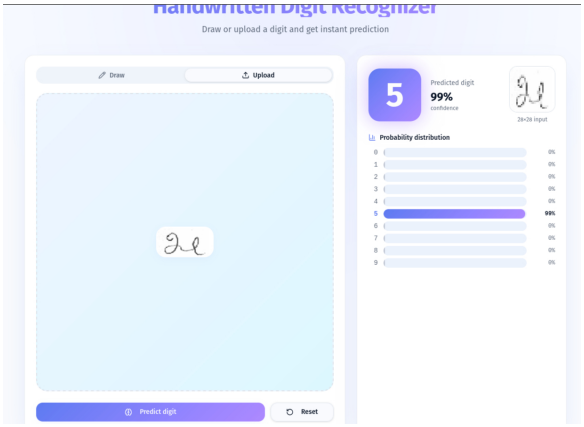
(a) Digit “7” predicted with 98% confidence



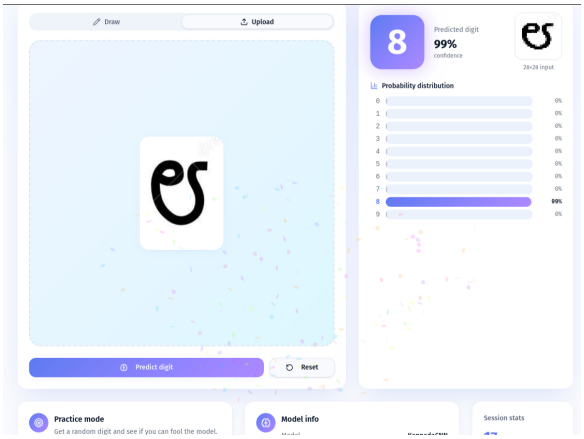
(b) Digit “8” predicted with 98% confidence



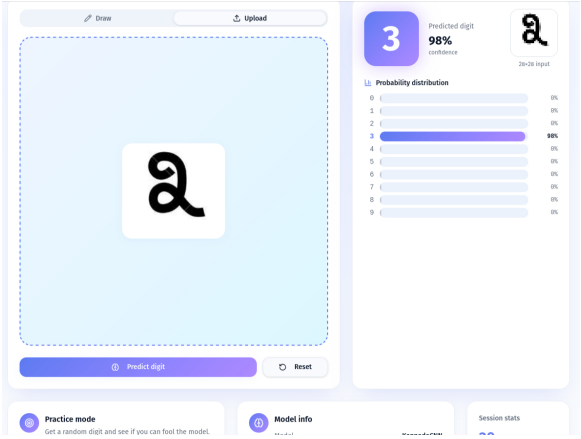
(a) Digit “9” predicted with 98% confidence



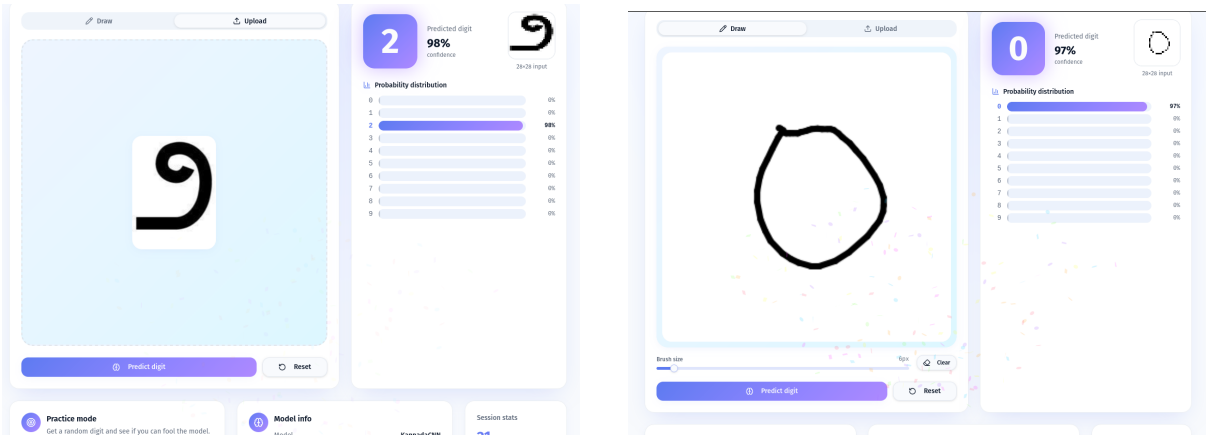
(b) Digit “5” predicted with 99% confidence



(a) Digit “8” predicted with 99% confidence



(b) Digit “3” predicted with 98% confidence



(a) Digit "2" predicted with 98% confidence (b) Digit "0" predicted with 97% confidence

Figure 6

9. Conclusion

We successfully built an end-to-end Kannada handwritten digit recognition system. A compact 3-layer CNN (∼290k parameters) trained from scratch on 60,000 samples achieves **99.67% validation accuracy** in under 5 minutes on a T4 GPU, surpassing the 97%+ target set in the assignment. The model was exported to ONNX and deployed in a modern, responsive React web application that supports free-hand drawing, image uploads, a probability distribution display, and a Practice Mode - all running client-side with zero server latency.

The project demonstrates that for well-structured digit classification problems, small CNNs trained from scratch with appropriate regularization and augmentation are not only sufficient but highly competitive.

10. Acknowledgements

LLM Usage: We used Claude for LaTeX report refining (to correct syntactical and grammatical error). GitHub Copilot assisted with TypeScript boilerplate in the React frontend.

Dataset: Kannada-MNIST, Kaggle, 2019. kaggle.com/competitions/Kannada-MNIST